Ex. No. : 06 **Date:**

Register No : 231701043 **Name: Ragul M**

A python program to do face recognition using SVM classifier

Aim:

To implement a python program to do face recognition using SVM classifier.

Requirements:

Jupyter Notebook with Python.

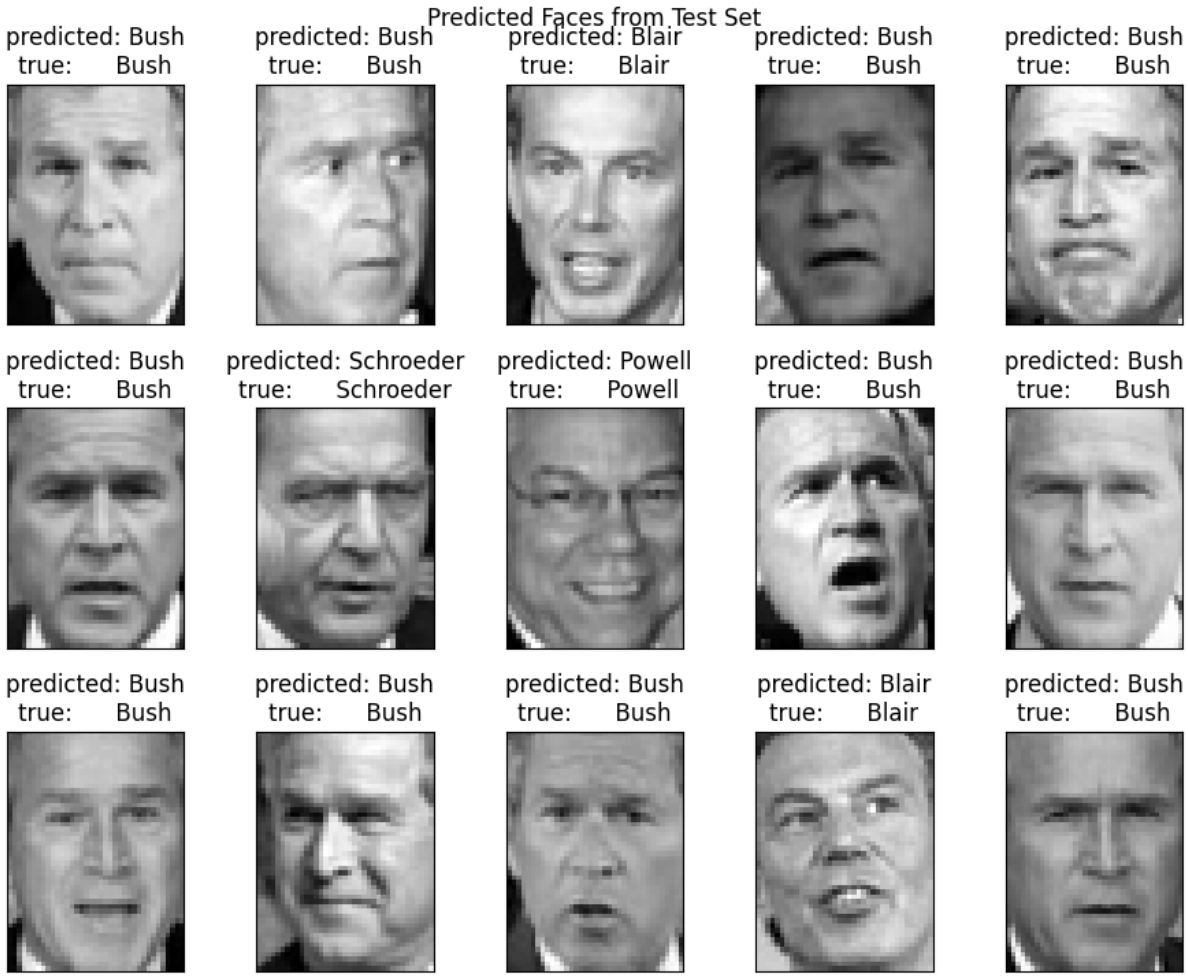
PROGRAM:

```
from sklearn.datasets import fetch_lfw_people
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import numpy as np

print("Loading Labeled Faces in the Wild™ dataset (this may take a moment)...")

lfw_people = fetch_lfw_people(min_faces_per_person=70, resize=0.4)

# Introspect the data: image shape, target names
```

```
n_samples, h, w = lfw_people.images.shape

X = lfw_people.data

n_features = X.shape[1]

y = lfw_people.target

target_names = lfw_people.target_names

n_classes = target_names.shape[0]


print(f"Total dataset size:")

print(f"n_samples: {n_samples}")

print(f"n_features: {n_features}")

print(f"n_classes: {n_classes}")


X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.25, random_state=42

)


print(f"Training set size: {X_train.shape[0]} samples")

print(f"Testing set size: {X_test.shape[0]} samples")


scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)


print("Extracting the top N 'eigenfaces' from the training set...")

n_components = 150
```

```
pca = PCA(n_components=n_components, svd_solver='randomized', whiten=True, random_state=42)

pca.fit(X_train_scaled)


eigenfaces = pca.components_.reshape((n_components, h, w))


X_train_pca = pca.transform(X_train_scaled)

X_test_pca = pca.transform(X_test_scaled)


print(f"Reduced dimensions from {n_features} to {n_components}")


print("Fitting the classifier to the training set...")

clf = SVC(kernel='rbf', C=1000, gamma=0.001, random_state=42)

clf.fit(X_train_pca, y_train)

print("Classifier training complete.")


print("Predicting people's names on the test set...")

y_pred = clf.predict(X_test_pca)


print("\nClassification Report:")

print(classification_report(y_test, y_pred, target_names=target_names))


print("\nConfusion Matrix:")

print(confusion_matrix(y_test, y_pred, labels=range(n_classes)))
```

```
def plot_gallery(images, titles, h, w, n_row=3, n_col=4):
    plt.figure(figsize=(1.8 * n_col, 2.4 * n_row))
    plt.subplots_adjust(bottom=0, left=.01, right=.99, top=.90, hspace=.35)
    for i in range(n_row * n_col):
        plt.subplot(n_row, n_col, i + 1)
        plt.imshow(images[i].reshape((h, w)), cmap=plt.cm.gray)
        plt.title(titles[i], size=12)
        plt.xticks(())
        plt.yticks(())


def title(y_pred, y_test, target_names, i):
    pred_name = target_names[y_pred[i]].rsplit(' ', 1)[-1]
    true_name = target_names[y_test[i]].rsplit(' ', 1)[-1]
    return 'predicted: %s\ntrue:      %s' % (pred_name, true_name)


prediction_titles = [title(y_pred, y_test, target_names, i) for i in range(y_pred.shape[0])]


plot_gallery(X_test, prediction_titles, h, w, n_row=3, n_col=5)
plt.suptitle('Predicted Faces from Test Set')


plot_gallery(eigenfaces, [f'eigenface {i}' for i in range(eigenfaces.shape[0])], h, w)
plt.suptitle('Eigenfaces (First 12 components)')


plt.show()
```



Output:

Loading Labeled Faces in the Wild™ dataset (this may take a moment)...

Total dataset size:

n_samples: 1288

n_features: 1850

n_classes: 7

Training set size: 966 samples

Testing set size: 322 samples

Extracting the top N 'eigenfaces' from the training set...

Reduced dimensions from 1850 to 150

Fitting the classifier to the training set...

Classifier training complete.

Predicting people's names on the test set...

Classification Report:

	precision	recall	f1-score	support
Ariel Sharon	0.64	0.69	0.67	13
Colin Powell	0.75	0.85	0.80	60
Donald Rumsfeld	0.73	0.70	0.72	27
George W Bush	0.90	0.90	0.90	146
Gerhard Schroeder	0.80	0.80	0.80	25
Hugo Chavez	0.73	0.53	0.62	15
Tony Blair	0.94	0.83	0.88	36
accuracy			0.84	322
macro avg	0.78	0.76	0.77	322
weighted avg	0.84	0.84	0.83	322

Confusion Matrix:

```
[[ 9  1  2  1  0  0  0]
 [ 2 51  3  2  1  1  0]
 [ 3  1 19  3  0  1  0]
 [ 0  9  2 132  2  1  0]
```

[0 1 0 3 20 0 1]

[0 3 0 2 1 8 1]

[0 2 0 3 1 0 30]] **Result:**

Thus, python program to do face recognition using SVM classifier has been successfully executed